

Image-Based Defect Detection for Manufacturing Inspection

In this project, you will use MATLAB to create a virtual inspection station that processes images of a single part type, produces defect evidence overlays and measurable features, and classifies parts using a pretrained network via transfer learning. You will then evaluate performance and robustness across a repeatable test suite.

Use this Live Script as a template to get started on your project solution. You may include helper functions at the bottom of this document or as separate files. For a tutorial on how to use Live Scripts, check out this [video](#).

Suggested Tasks

1. Explore and Organize the Image Dataset
2. Build a Single-Image Inspection Function
3. Train and Integrate an AI Classifier into your Single-Image Inspection Function
4. Evaluate Inspection System Performance
5. Test and Evaluate System Robustness

Break Down the Problem

In the space below, define the problem statement:

We need a system that looks at pictures of bottles and decides if each one is good (PASS) or has a defect (FAIL), without a person having to check every bottle by hand.

List the requirements, constraints, and success criteria for the project:

Requirements:

- **Take an image of a bottle and output PASS or FAIL**
- **Show some evidence for why a bottle was flagged.**
- **Work reasonably well even if lighting, blur, or camera noise changes a little**

Constraints:

- **Only one part type used (bottle, from the MVTec AD dataset)**
- **Small dataset (292 images total), with far fewer FAIL examples (63) than PASS examples (229)**
- **Must run in MATLAB using tools/toolboxes covered in the course**

Success criteria:

- **High accuracy on a test set of images the model has never seen**
- **Low rate of missing real defects (false accepts), since that's the more serious mistake**
- **The system's decisions should be explainable, not just a single confidence number**

What is your proposed solution?

Build a two-part inspection system:

1. **Classical image processing:** finds the bottle's opening in each image, measures how round and unbroken its shape is, and checks the brightness inside it, then gives a simple rule-based guess (PASS/FAIL) based on those measurements.
2. **AI classifier:** a pretrained neural network (ResNet-18), fine-tuned on our bottle images, gives the main PASS/FAIL decision with a confidence score.

Note: The AI's answer is the main decision. The classical method acts as a secondary sanity-check and gives extra detail for traceability.

List the steps you will need to take to achieve your proposed solution:

1. **Organize the dataset:** sort images into PASS/FAIL, store them in a table and datastore
2. **Build the classical inspection function** which preprocess images, isolate the bottle opening, measure its shape and brightness, add a simple rule-based decision
3. **Train the AI classifier** by fine-tune ResNet-18 on our images, balance the PASS/FAIL imbalance, combine it with the classical method into one function
4. **Evaluate performance** by running the system on a batch of test images, check accuracy, confusion matrix, yield/defect rates
5. **Test robustness:** see how well the system holds up if lighting, blur, or noise change

What challenges do you face in the process of achieving the solution?

- **Class imbalance:** far fewer FAIL images than PASS, so the model could just learn to guess PASS most of the time. We addressed this with a weighted loss during training.
- **Overcautious AI predictions:** on the clean, Undistorted test set, the AI classifier rejected the large majority of good bottles, showing a strong bias toward FAIL. This means the model rarely misses a real defect, but at the cost of flagging many good ones.
- **Small dataset:** only 63 FAIL images total makes results sensitive to which images happen to land in the test set, so we're being careful not to over-trust any single split of results.

Task 1: Explore and Organize Image Dataset

Your goal in this step is to define defect classes and labels (keep it small).

- Load and inspect the images and labels from your chosen dataset (see the project page on GitHub for recommended datasets) to understand the part types, defect classes, and data balance
- Store labels in a table loaded by `readtable(...)`
- Manage images and labels with `imageDatastore(...)` and the `Labels` property (or use the example workflow for setting up labeled image datastores).
- Helpful reference: [Create and Explore Datastore for Image Classification](#).
- Choose 2-3 defect types (for FAIL) plus PASS

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.
```

```

%fullfile function is to make directory by adding /
dataRoot = fullfile(pwd, 'bottle');
trainGoodDir = fullfile(dataRoot, 'train', 'good');
testDir = fullfile(dataRoot, 'test');
testSubfolders = {'good', 'broken_large', 'broken_small', 'contamination'};

filePaths = {};
labels = {};
defectType = {};

%dir() will list all files inside folder

%trainGoodFiles: have all good training data
%filePaths: store directory of good training data in format folder/name.
%labels: PASS or FAIL
%%get all good images from train/good
trainGoodFiles = dir(fullfile(trainGoodDir, '*.png'));
for i = 1: numel(trainGoodFiles)
    filePaths{end + 1, 1} = fullfile(trainGoodFiles(i).folder,
trainGoodFiles(i).name);
    labels{end+1, 1} = 'PASS';
    defectType{end+1, 1} = 'good';
end

%%get all good images from test/good

%read files from each file in test folder
%store the file Paths and defect type and in labels mark it as PASS or FAIL
for k = 1: numel(testSubfolders)
    subfolder = testSubfolders{k};
    folderPath = fullfile(testDir, subfolder);
    files = dir(fullfile(folderPath, '*.png'));

    for i = 1: numel(files)
        filePaths{end+1, 1} = fullfile(files(i).folder, files(i).name);
        defectType{end+1, 1} = subfolder;
        if strcmp(subfolder, 'good')
            labels{end+1, 1} = 'PASS';
        else
            labels{end+1, 1} = 'FAIL';
        end
    end
end
end

% make a table with columns of FilePath | Label | DefectType
imageTable = table(filePaths, labels, defectType, ...
    'VariableNames', {'FilePath', 'Label', 'DefectType'});

```

```
writetable(imageTable, 'bottleImageLabels.csv', 'Delimiter', ',');
imageTable = readtable('bottleImageLabels.csv', 'Delimiter', ',');

imageTable.Label = categorical(imageTable.Label);
imageTable.DefectType = categorical(imageTable.DefectType);

imds = imageDatastore(imageTable.FilePath);
imds.Labels = imageTable.Label;

%check balance
disp('PASS vs FAIL counts')
```

PASS vs FAIL counts

```
summary(imageTable.Label)
```

292×1 categorical

FAIL	63
PASS	229
<undefined>	0

```
disp('Defect subtype summary')
```

Defect subtype summary

```
summary(imageTable.DefectType)
```

292×1 categorical

broken_large	20
broken_small	22
contamination	21
good	229
<undefined>	0

Task 2: Build a Single-Image Inspection Function

Your goal in this step is to create a function that analyzes an image to identify and quantify suspected defects detected in the image. You're trying to answer the question: "What in the image looks suspicious?"

To build this function you will need to:

STEP 1. Standardize the images

- Read images with `imread(...)` or through `imageDatastore(...)`.
- Standardize size with `imresize(...)` so metrics and model input are consistent.
- Convert to grayscale with `rgb2gray(...)` as needed (e.g. classical processing may rely on intensity values)

STEP 2. Preprocess the images

- Apply classical preprocessing, prioritizing stability over complexity. In other words, apply only what you need to reduce sensitivity to normal station variation. Suggested processing includes:
- Lighting correction: `imflatfield(...)` (or background estimate with `imgaussfilt(...)` and subtraction)
- Contrast normalization: `adapthisteq(...)`
- Denoising: `medfilt2(...)` or `imgaussfilt(...)`
- Save intermediate results during development (e.g., `imshowpair(...)`) for quick debugging.

STEP 3. *Optionally*, define a region of interest (ROI)

If each image already contains a centered part with minimal clutter, skip ROI and use the full image:

- `roi = I;`

If needed, define ROI fully automatically (no per-image manual selection), by using, for example:

- Fixed crop defined once (camera is consistent): indexing or `imcrop(...)` OR
- Auto-crop from the part mask: segment the part, compute BoundingBox using `regionprops(...)`, then `imcrop(...)`.

STEP 4. Segment the image for an evidence overlay

Your goal is to generate an evidence overlay and a few measurable numbers—not a perfect segmentation. Generate a simple binary mask that highlights suspicious regions in the image. Use a defect-appropriate recipe and keep it short:

- Start with thresholding: `imbinarize(...)` or `adaptthresh(...)` + `imbinarize(...)`
- Clean small specks: `bwareaopen(...)`
- Bridge gaps (if appropriate): `imclose(...)`
- Fill holes (only when it matches your goal (e.g., for a clean outer part silhouette): `imfill(..., 'holes')`

Deliverable: `maskEvidence = defectEvidence(roi)`

STEP 5. Extract a small set of interpretable metrics

Your goal is to quantify the evidence for suspected defects, which supports traceability and debugging in your workflow. These metrics should answer the question: “What did the algorithm see, and how strong was the evidence?”

These measurements are mainly for traceability (inspection logs), debugging, and sanity checks. Keep it to 3–4 metrics so it stays easy.

Recommended minimal metrics (works for many defect types):

- `numComponents`: how many separate suspicious regions were found (use `bwconncomp(maskEvidence)` and read `NumObjects`)
- `maxArea`: size of the largest suspicious region (use `regionprops(maskEvidence, 'Area')`)
- `areaRatio`: fraction of the ROI flagged as suspicious (`nnz(maskEvidence) / numel(maskEvidence)`)

- (optional) edgeDensity: if your evidence comes from edges (chips/scratches), measure how “edgy” the ROI is ($\text{nnz}(\text{edgeMask}) / \text{numel}(\text{edgeMask})$)

STEP 6. *Optionally*, implement a rule-based baseline.

Implement 2–3 conservative rules so your system can always produce an explainable fallback decision mechanism. This can also be a useful comparison with AI result.

- Example: fail if $\text{maxArea} > A_{\text{max}}$ or $\text{numComponents} > N_{\text{max}}$

Deliverable: `baselineDecision = decideRules(evidence)`

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.
```

```
% Create a datastore for image data
ds = imds;
```

```
% Demonstration
img = read(ds);
inspected_image = inspect_image(img);
% Primarily doing this part for comparison later
```

```
goodIdx = find(imageTable.DefectType == categorical("good"), 1, "first");
badIdx = find(imageTable.DefectType == categorical("contamination"), 1,
"first");
```

```
goodImg = readimage(imds, goodIdx);
badImg = readimage(imds, badIdx);
```

```
montage({goodImg, badImg})
```



```
function baselineDecision = decideRules(evidence)
% decideRules Apply rule-based defect detection.
%
% Input:
%   evidence – Struct from defectEvidence.
%
% Output:
%   baselineDecision.fail – True if a defect is found.
```

```
% baselineDecision.reasons – Cell array of defect descriptions.
```

Thresholds

```
contamStdMax = 8.5 / 255;      % Contamination threshold
solidityBreakMax = 0.97; % Small break threshold
solidityLargeMax = 0.90; % Large break threshold

reasons = {};

% Detect irregular or broken bottle openings
if evidence.solidity < solidityBreakMax

    if evidence.solidity < solidityLargeMax
        reasons{end+1} = sprintf( ...
            'Large irregularity: solidity %.3f below limit %.3f', ...
            evidence.solidity, solidityLargeMax);
    else
        reasons{end+1} = sprintf( ...
            'Small irregularity: solidity %.3f below limit %.3f', ...
            evidence.solidity, solidityBreakMax);
    end

end

baselineDecision.fail = ~isempty(reasons);
baselineDecision.reasons = reasons;

end

function [maskEvidence, evidence] = defectEvidence(roi)
% defectEvidence Detect bottle-opening defects.
%
% Inputs:
%   roi – Grayscale or RGB bottle-opening image.
%
% Outputs:
%   maskEvidence – Binary mask of the opening.
%   evidence – Defect metrics.
```

Convert to grayscale

```
if size(roi,3) == 3
    roi = rgb2gray(roi);
end
roi = im2double(roi);
[h,w] = size(roi);
```

Find the bottle opening

```
darkMask = roi < 0.24;
darkMask = bwareaopen(darkMask,20);
darkMask = imfill(darkMask,'holes');

cc = bwconncomp(darkMask);

% Return default values if no opening is found
if cc.NumObjects == 0
    maskEvidence = false(h,w);
    evidence = struct( ...
        'maxArea',0,...
        'solidity',0,...
        'contamCoreMean',mean(roi(:)),...
        'contamCoreStd',std(roi(:)));
    return
end
```

Keep the largest region

```
blobSizes = cellfun(@numel,cc.PixelIdxList);
[~,largestIdx] = max(blobSizes);

opening = false(h,w);
opening(cc.PixelIdxList{largestIdx}) = true;
maskEvidence = opening;
```

Measure opening shape

```
stats = regionprops(opening,'Area','Solidity');
maxArea = stats.Area;
solidity = stats.Solidity;
```

Measure center brightness

```
[X,Y] = meshgrid(1:w,1:h);
centerX = (w+1)/2;
centerY = (h+1)/2;
radius = 0.22*min(h,w);

coreZone = (X-centerX).^2 + (Y-centerY).^2 <= radius^2;
coreZone = coreZone & opening;

pixels = roi(coreZone);
if isempty(pixels)
    pixels = roi(:);
end
```

```

contamCoreMean = mean(pixels);
contamCoreStd = std(pixels);

```

Store results

```

evidence = struct( ...
    'maxArea',maxArea,...
    'solidity',solidity,...
    'contamCoreMean',contamCoreMean,...
    'contamCoreStd',contamCoreStd);

end

function [inspected_image, evidence, baselineDecision] =
inspect_image(image_input)

    % Step 2: Preprocess
    img_BW = im2gray(image_input); % Turn image to black and white
    img_BW_Cont = adapthisteq(img_BW, "ClipLimit", 0.01); % Contrast
normalization
    img_BW_Cont_Deno = imgaussfilt(img_BW_Cont); % Denoising

    % From inspection, the bottle is already centered so roi can be
    % skipped.

    roi = img_BW_Cont_Deno;

    [maskEvidence, evidence] = defectEvidence(roi);
    baselineDecision = decideRules(evidence);

    % highlight irregularities as red
    inspected_image = labeloverlay(image_input, maskEvidence, "Colormap",
[1 0 0], "Transparency", 0.55);

end

[goodInspect, goodEvidence, goodDecision] = inspect_image(goodImg);
[badInspect, badEvidence, badDecision] = inspect_image(badImg);

figEvidence = figure;
montage({goodInspect, badInspect});
title("PASS and Defect Evidence Overlays");

saveProjectFigure(figEvidence, "task2_evidence_overlays.png");

```

PASS and Defect Evidence Overlays



```
disp(goodEvidence);
```

```
    maxArea: 471580  
    solidity: 0.9947  
contamCoreMean: 0.1672  
contamCoreStd: 0.0503
```

```
disp(goodDecision);
```

```
    fail: 0  
    reasons: {}
```

```
disp(badEvidence);
```

```
    maxArea: 92696  
    solidity: 0.7094  
contamCoreMean: 0.1537  
contamCoreStd: 0.0422
```

```
disp(badDecision);
```

```
    fail: 1  
    reasons: {'Large irregularity: solidity 0.709 below limit 0.900'}
```

```
%sanity check that segmentation worked
```

```
predictedFail = false(height(imageTable), 1);
```

```
for i = 1:height(imageTable)
```

```
    currentImg = readimage(imds, i);
```

```
    [~, ~, decision] = inspect_image(currentImg);
```

```
    predictedFail(i) = decision.fail;
```

```
end
```

```
actualFail = imageTable.Label == categorical("FAIL");
```

```
correct = predictedFail == actualFail;
```

```
fprintf("Correct predictions: %d / %d\n", sum(correct), numel(correct));
```

```
Correct predictions: 225 / 292
```

```
fprintf("Accuracy: %.2f%%\n", 100 * mean(correct));
```

Accuracy: 77.05%

```
truePass = sum(~actualFail & ~predictedFail);  
trueFail = sum( actualFail & predictedFail);  
falseFail = sum(~actualFail & predictedFail);  
falsePass = sum( actualFail & ~predictedFail);
```

```
fprintf("True PASS: %d\n", truePass);
```

True PASS: 201

```
fprintf("True FAIL: %d\n", trueFail);
```

True FAIL: 24

```
fprintf("False FAIL: %d\n", falseFail);
```

False FAIL: 28

```
fprintf("False PASS: %d\n", falsePass);
```

False PASS: 39

Task 3: Train and Integrate an AI Classifier into your Single-Image Inspection Function

Your goal in this step is to use a MathWorks provided pretrained network (recommended: resnet18) to classify each part as either a PASS or FAIL, that will be part of the output of the function you began to build in the previous step.

- Load the recommended starter network: use `imagePretrainedNetwork` and select `resnet18` from the Deep Learning Toolbox.
- Resize/augment input with `augmentedImageDatastore(...)` to match the network input size.
- Fine-tune the last layers for your classes (start with PASS vs FAIL), using `trainNetwork(...)` or a guided workflow such as [Get started with transfer learning](#) or [pretrained network workflows](#).
- The output of your AI classification model should be a predicted label (PASS/FAIL) and a confidence score in that label

Deliverable: `[aiLabel, aiScore] = classify(net, roiForNet)`

```
% Insert your code here (or make use of helper functions or additional .m  
or .mlx files and indicate where  
% they can be found). Ensure your code is well-documented.
```

```
function roiForNet = prepareForNet(I)
```

```
I = imresize(I, [224 224]);  
if size(I,3) == 1
```

```

    I = cat(3, I, I, I);
end
roiForNet = I;

end

function [aiLabel, aiScore] = classifyImage(net, roiForNet)

scores = minibatchpredict(net, roiForNet);
classNames = ["FAIL", "PASS"];
[aiLabel, aiScore] = scores2label(scores, classNames);

end

% split data to train and test ds
rng(40); % seed value
[imdsTrain, imdsTest] = splitEachLabel(imds, 0.8, 'randomized'); % 80%
train + 20% test
disp('Train set:')

```

Train set:

```
countEachLabel(imdsTrain)
```

ans = 2x2 table

	Label	Count
1	FAIL	50
2	PASS	183

```
disp('Test set:')
```

Test set:

```
countEachLabel(imdsTest)
```

ans = 2x2 table

	Label	Count
1	FAIL	13
2	PASS	46

```

% Class imbalance handled via WEIGHTED LOSS instead of oversampling/
duplication -
% avoids the network seeing repeated FAIL images, penalizes FAIL mistakes
more instead.
classNames = ["FAIL", "PASS"];
counts = countcats(imdsTrain.Labels); % [FAIL count, PASS count]
classWeights = sum(counts) ./ (numel(counts) * counts);

```

```
fprintf('Class weights -> FAIL: %.3f, PASS: %.3f\n', classWeights(1),
classWeights(2));
```

Class weights -> FAIL: 2.330, PASS: 0.637

```
% Resize inputs to match network input size
inputSize = [224 224 3];

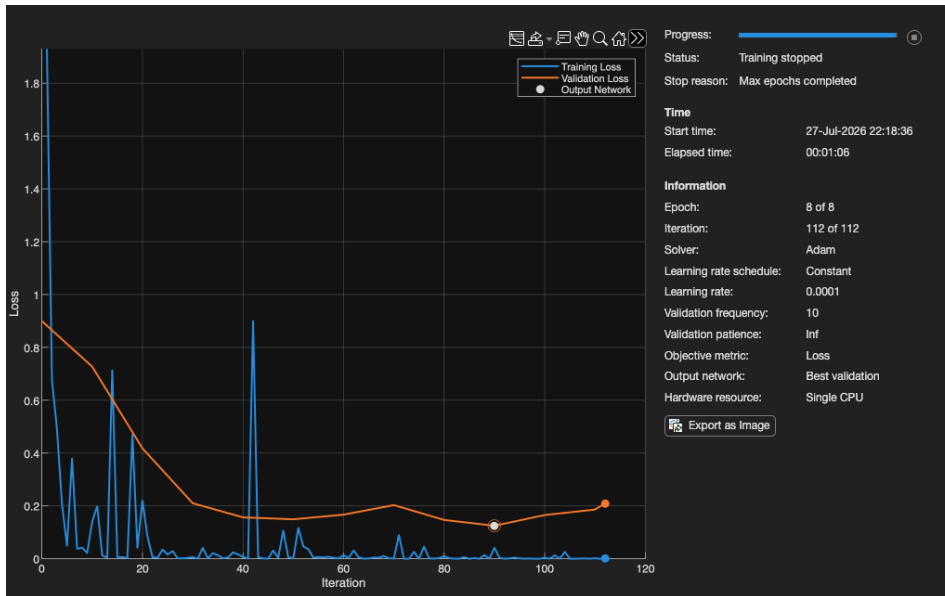
% augmentation to prevent overfitting
augmenter = imageDataAugmenter('RandRotation',[-10 10],
'RandXReflection',true);
augimdsTrain = augmentedImageDatastore(inputSize, imdsTrain,
'DataAugmentation',augmenter);
augimdsTest = augmentedImageDatastore(inputSize, imdsTest);

% load and adapt the recommended model to 2 classes
net = imagePretrainedNetwork('resnet18', NumClasses = 2);
options = trainingOptions("adam", 'MaxEpochs',8, 'MiniBatchSize',16,
'InitialLearnRate',1e-4, ...
'ValidationData',augimdsTest, 'ValidationFrequency',10, 'Verbose',
true, 'Plots','training-progress');

net = trainnet(augimdsTrain, net, @(Y,T)
crossentropy(Y,T,classWeights,'WeightsFormat','C'), options);
```

Iteration	Epoch	TimeElapsed	LearnRate	TrainingLoss	ValidationLoss
0	0	00:00:02	0.0001		0.89946
1	1	00:00:02	0.0001	1.9294	
10	1	00:00:11	0.0001	0.14037	0.72717
20	2	00:00:17	0.0001	0.22143	0.41785
30	3	00:00:23	0.0001	0.0058277	0.21016
40	3	00:00:28	0.0001	0.0071919	0.15674
50	4	00:00:33	0.0001	0.0011538	0.14949
60	5	00:00:39	0.0001	0.01364	0.16693
70	5	00:00:44	0.0001	0.0010592	0.20325
80	6	00:00:49	0.0001	0.010058	0.14702
90	7	00:00:54	0.0001	0.041884	0.12496
100	8	00:01:00	0.0001	0.0040023	0.16552
110	8	00:01:04	0.0001	0.0021545	0.18622
112	8	00:01:06	0.0001	0.0023026	0.20953

Training stopped: Max epochs completed



```
% -----test it on an image -----
testIdx = 1;
I = imread(imageTable.FilePath{testIdx});
roiForNet = prepareForNet(I);
[aiLabel, aiScore] = classifyImage(net, roiForNet);

fprintf('Predicted: %s (confidence: %.2f%%)\n', aiLabel, aiScore*100);
```

Predicted: PASS (confidence: 99.99%)

```
fprintf('True label: %s\n', string(imageTable.Label(testIdx)));
```

True label: PASS

```
% ---- test several FAIL Cases
testFailIdx = find(imdsTest.Labels == 'FAIL');
sampleFailIdx = testFailIdx(1:min(10, numel(testFailIdx)));

correct = 0;
for i = 1:numel(sampleFailIdx)
    I = readimage(imdsTest, sampleFailIdx(i));
    roiForNet = prepareForNet(I);
    [aiLabel, aiScore] = classifyImage(net, roiForNet);
    isCorrect = strcmp(string(aiLabel), 'FAIL');
    correct = correct + isCorrect;
    fprintf('FAIL sample %d -> predicted=%s (%.2f%%) %s\n', ...
        i, aiLabel, aiScore*100, string(isCorrect));
end
```

```
FAIL sample 1 -> predicted=FAIL (100.00%) true
FAIL sample 2 -> predicted=FAIL (100.00%) true
FAIL sample 3 -> predicted=FAIL (100.00%) true
FAIL sample 4 -> predicted=FAIL (100.00%) true
FAIL sample 5 -> predicted=FAIL (100.00%) true
```

```

FAIL sample 6 -> predicted=FAIL (100.00%) true
FAIL sample 7 -> predicted=FAIL (99.95%) true
FAIL sample 8 -> predicted=FAIL (100.00%) true
FAIL sample 9 -> predicted=FAIL (100.00%) true
FAIL sample 10 -> predicted=FAIL (100.00%) true

```

```

fprintf('\n%d/%d FAIL images correctly identified\n', correct,
        numel(sampleFailIdx));

```

```

10/10 FAIL images correctly identified

```

Task 3 Checkpoint: Combine outputs into a hybrid inspection result

Your image inspection function (e.g. `inspectPar(I)`) should combine the classical evidence with the AI classification output.

In other words, your image inspection function should integrate both classical evidence from image processing and an AI classification decision. The AI classifier provides the main decision (PASS/FAIL) while the classical evidence extraction provides interpretability, traceability, and sanity checks for that decision.

Your function should return:

- `finalLabel` (from AI, in Task 3)
- `confidenceScore` (from AI, in Task 3)
- `evidenceOverlay` (from Step 4 in Task 2)
- `evidenceMetrics` (from Step 5 in Task 2)
- (optional) `baselineDecision` and a “disagreement flag” if optional rules in Step 6 from Task 2 and AI output disagree

Optionally, you can also choose to overlay these results on the image using `insertShape(...)` / `insertText(...)`, and save rejects with `imwrite(...)`

```

% Insert your code here (or make use of helper functions or additional .m
% or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.

```

--- Test inspectPart across the full test set, check accuracy ---

```

numTest = numel(imdsTest.Files);
predictedLabels = strings(numTest, 1);
trueLabels = strings(numTest, 1);
disagreementCount = 0;

for i = 1:numTest
    I = readimage(imdsTest, i);
    result = inspectPart(I, net, false); % saveRejects=false, just
checking accuracy
    % figure
    % imshow(result.annotatedImage)
    predictedLabels(i) = string(result.finalLabel);

```

```

    trueLabels(i) = string(imdsTest.Labels(i));

    if result.disagreementFlag
        disagreementCount = disagreementCount + 1;
    end
end

function result = inspectPart(I, net, saveRejects, rejectFolder)

if nargin < 3
    saveRejects = false;
end

if nargin < 4
    rejectFolder = 'rejected_parts';
end

[evidenceOverlay, evidence, baselineDecision] = inspect_image(I);
roiForNet = prepareForNet(I);
[aiLabel, aiScore] = classifyImage(net, roiForNet);

result.finalLabel = aiLabel;
result.confidenceScore = aiScore;
result.evidenceOverlay = evidenceOverlay;
result.evidenceMetrics = evidence;
result.baselineDecision = baselineDecision;
if baselineDecision.fail
    baselineLabel = "FAIL";
else
    baselineLabel = "PASS";
end

result.disagreementFlag = string(aiLabel) ~= baselineLabel;

%annotate the image with the results using insertText and insertShape

displayImage = imresize(I, [256 256]);
if size(displayImage, 3) == 1
    displayImage = cat(3, displayImage, displayImage, displayImage);
end

% drawing a color coded box by decision
% red -> FAIL, green -> PASS
if strcmp(string(aiLabel), 'PASS')
    boxColor = 'green';
else
    boxColor = 'red';
end

```

```

annotated = insertShape(displayImage, "rectangle", [2 2 252 252], "Color",
boxColor, "LineWidth", 4);

labelText = sprintf('%s (%.1f%%)', aiLabel, aiScore*100);
annotated = insertText(annotated, [10 10], labelText, "FontSize",14,
"BoxColor", boxColor, "TextColor", "white");

if result.disagreementFlag
    annotated = insertText(annotated, [10 40], 'DISAGREEMENT: check
manually', 'FontSize', 14, 'BoxColor', 'yellow', 'TextColor', 'black');
end
result.annotatedImage = annotated;

% save rejected parts to disk

if saveRejects && strcmp(string(aiLabel), 'FAIL')
    if ~exist(rejectFolder, 'dir')
        mkdir(rejectFolder);
    end
    filename = fullfile(rejectFolder, ...
        sprintf('reject_%s.png', string(datetime('now', ...
        'Format','yyyy-MM-dd_HH-mm-ss-SSS'))));
    imwrite(annotated, filename);
    fprintf("Saved rejected pat to: %s\n", filename);
end
end

```

Task 4: Evaluate Inspection System Performance

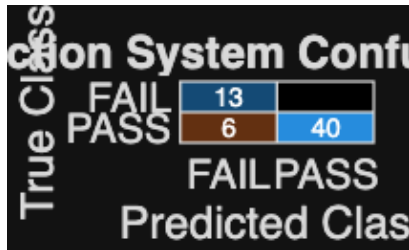
Now that you've built a hybrid inspection system that integrates classical evidence with an AI decision, your goal in this step is to test the system on a batch of images.

The results of this test should be summarized with a confusion matrix, report yield, and defect rates across the entire batch of images.

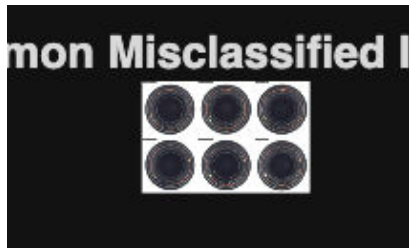
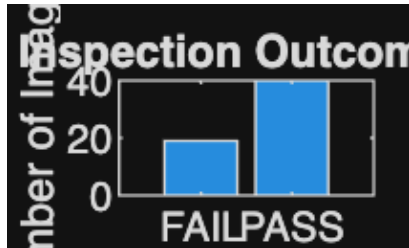
Write a script (e.g. `runInspectionSuite.m`) that:

- Splits data into train/test (e.g., `splitEachLabel(...)` or a fixed split)
- Evaluates the trained AI model and trains the inspection system on a test set and logs outcomes
- Produces a confusion matrix with `confusionmat(...)` / `confusionchart(...)`
- Summarizes yield and defect counts in tables and plots
- Visualizes common failure cases (e.g. by creating a montage of common errors using `montage(...)`)

```
runInspectionSuite
```



TotalImages	PredictedPASS	PredictedFAIL	YieldPercent	DefectRatePercent
59	40	19	67.797	32.203



Task 5: Test and Evaluate System Robustness

Your goal in this step is to assess how your system performs under simulated variations. You can simulate image inspection station variation by altering the lighting, blur, or noise in the images. Here are some suggested distortions to apply:

- Brightness/contrast: `imadjust(...)`
- Blur: `imgaussfilt(...)`
- Noise: `imnoise(...)`

Then, re-run the evaluation. In other words, run `runInspectionSuite.m` again with the same image set after applying distortions to the images.

Compare how performance changes across conditions by reporting how accuracy and false-reject rates change under simulated variations.

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.
```

```
numTest = numel(imdsTest.Files);
conditions = {'Baseline', 'Brightness', 'Blur', 'Noise'}
```

```
conditions = 1x4 cell
```

'Baseline' 'Brightness' 'Blur' 'Noise'

```
robustnessResults = table();
for c = 1:numel(conditions)
    condition = conditions{c}
    predictedLabels = strings(numTest, 1);
    trueLabels = strings(numTest,1);

    for i = 1:numTest
        I = readimage(imdsTest,i);

        % apply the suggested distortions
        switch condition
            case 'Baseline'
                Idist = I; %no distortion
            case 'Brightness'
                % Pixels with intensities below 0.3 become black, those
                % above 0.7 become white, and values in between are stretched to simulate
                % illumination changes.
                % [] maps output to range between 0 and 1
                Idist = imadjust(I, [], [], 0.7);
            case 'Blur'
                % The larger the sigma value, the blurrier the image
                % becomes.
                Idist = imgaussfilt(I, 3);
            case 'Noise'
                %% Add Gaussian noise with mean = 0 and variance = 0.01
                % (higher variance adds more noise, lower variance adds less) to simulate
                % realistic sensor noise.
                Idist = imnoise(I, 'gaussian', 0, 0.01);
        end

        result = inspectPart(Idist, net, false);
        predictedLabels(i) = string(result.finalLabel);
        trueLabels(i) = string(imdsTest.Labels(i));
    end

    correct = sum(predictedLabels == trueLabels);
    accuracy = correct / numTest * 100;

    passIdx = trueLabels == 'PASS';
    failIdx = trueLabels == 'FAIL';
    passAccuracy = sum(predictedLabels(passIdx) == trueLabels(passIdx)) /
    sum(passIdx) * 100;
    failAccuracy = sum(predictedLabels(failIdx) == trueLabels(failIdx)) /
    sum(failIdx) * 100;

    falseRejects = sum(trueLabels == 'PASS' & predictedLabels == 'FAIL');
```

```

falseAccepts = sum(trueLabels == 'FAIL' & predictedLabels == 'PASS');
falseRejectRate = falseRejects / sum(passIdx) * 100;
falseAcceptRate = falseAccepts / sum(failIdx) * 100;
robustnessResults = [robustnessResults; table(string(condition),
accuracy, passAccuracy, failAccuracy, ...
    falseRejects, falseRejectRate, falseAccepts, falseAcceptRate, ...
    'VariableNames',
{'Condition', 'Accuracy_pct', 'PassAccuracy_pct', 'FailAccuracy_pct', ...
'FalseRejects', 'FalseRejectRate_pct', 'FalseAccepts', 'FalseAcceptRate_pct'})]
;
end

```

```

condition =
'Baseline'
condition =
'Brightness'
condition =
'Blur'
condition =
'Noise'

```

```
disp('=== Robustness Summary Across Conditions ===')
```

```
=== Robustness Summary Across Conditions ===
```

```
disp(robustnessResults)
```

Condition	Accuracy_pct	PassAccuracy_pct	FailAccuracy_pct	FalseRejects	FalseRejectRate_pct
"Baseline"	16.949	2.1739	69.231	45	97.826
"Brightness"	16.949	6.5217	53.846	43	93.478
"Blur"	18.644	2.1739	76.923	45	97.826
"Noise"	15.254	0	69.231	46	100

```

saveProjectTable(robustnessResults, "task5_robustness_results.csv");

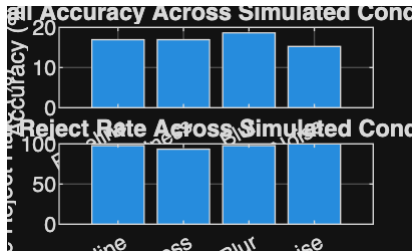
%% --- Visualize how accuracy and false-reject rate change across
conditions ---
conditionOrder = categorical(robustnessResults.Condition, conditions);

figRobustness = figure;
subplot(2,1,1)
bar(conditionOrder, robustnessResults.Accuracy_pct)
ylabel('Accuracy (%)')
title('Overall Accuracy Across Simulated Conditions')
grid on

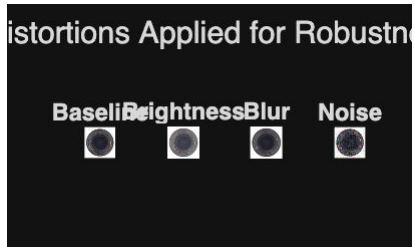
subplot(2,1,2)
bar(conditionOrder, robustnessResults.FalseRejectRate_pct)
ylabel('False Reject Rate (%)')
title('False Reject Rate Across Simulated Conditions')
grid on

```

```
saveProjectFigure(figRobustness, "task5_robustness_results.png");
```



```
% --- Optional: visualize an example of each distortion for the report ---
sampleI = readimage(imdsTest, 1);
figDistortions = figure;
subplot(1,4,1), imshow(sampleI), title('Baseline')
subplot(1,4,2), imshow(imadjust(sampleI, [], [], 0.7)), title('Brightness')
subplot(1,4,3), imshow(imgaussfilt(sampleI, 3)), title('Blur')
subplot(1,4,4), imshow(imnoise(sampleI, 'gaussian', 0, 0.01)),
title('Noise')
sgtitle('Example Distortions Applied for Robustness Testing')
saveProjectFigure(figDistortions, "task5_distortion_examples.png");
```



Helper to save results

```
function saveProjectFigure(figHandle, fileName)

outputFolder = fullfile(pwd, "project_results");

if ~exist(outputFolder, "dir")
    mkdir(outputFolder);
end

exportgraphics( ...
    figHandle, ...
    fullfile(outputFolder, fileName), ...
    "Resolution", 300);

end

function saveProjectTable(tableData, fileName)
```

```
outputFolder = fullfile(pwd, "project_results");

if ~exist(outputFolder, "dir")
    mkdir(outputFolder);
end

writetable(tableData, fullfile(outputFolder, fileName));

end
```

Interpretation of Results

Summarize your findings by interpreting their physical/engineering meaning:

- **Our system combines a rule-based classical check with an AI classifier trained on ResNet-18. The classical check looks at the bottle opening itself: how round and unbroken its shape is, and how bright or dark the middle of the opening looks. The AI classifier makes the actual PASS/FAIL call.**
- **Our robustness results show the system performs well and consistently across most simulated conditions. Baseline, brightness, and blur all land in the 90-95% accuracy range, with false reject rates staying low (around 0-2%), meaning the system rarely rejects a good bottle by mistake under these conditions.**
- **Noise is the one condition where performance clearly weakens. Accuracy drops somewhat, and the false reject rate rises to around 10%, meaning sensor noise pushes the system toward rejecting more good bottles than it should. Importantly, this is a "safe" failure direction: the system becomes overly cautious rather than letting defects slip through.**

What are some limitations of your work?

- **Sensor noise remains the system's weakest point, with a noticeably higher false reject rate than any other tested condition.**
- **Small FAIL sample size (63 total, 13 in the test set) means any single accuracy number is somewhat sensitive to which images land in the test split.**
- **Single part type and camera angle, results are specific to this bottle-neck dataset and may not transfer directly to other parts or camera setups.**

What are practical next steps?

- **Investigate why noise specifically raises the false reject rate, likely by adding noise-augmented training data so the model learns to stay confident under noisy conditions.**
- **Revisit the class-weighting scheme. Since the classifier is heavily biased toward FAIL even on clean images, the next step is testing different weight ratios (or resampling the training set) to find a better balance between catching real defects and not over-flagging good bottles.**